

Lab-13.5

Course : AI Assisted Coding

Topic: Improving Legacy Code with AI Suggestions

Name : Mohammed Sabir

Hall Ticket No : 2303A51506

Batch : 22

Task-01

Final Optimal Prompt:

remove the global variable from given code

Code Screenshot :

```
5 def calculate_interest(amount, rate):  
6     return amount * rate  
7 print(calculate_interest(1000, 0.1))  
8
```

Output Screenshot:

```
● PS C:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant> & 'c:\python314\python.exe' -x64 -u -s -i 'c:\python314\python.exe' -- 'c:\python314\python.exe' 100.0  
○ PS C:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant>
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

This task focuses on improving code quality by avoiding unnecessary reliance on global variables. Although the given function already uses parameters effectively, the key concept is ensuring that all required data is passed explicitly to functions rather than being accessed globally. This approach enhances modularity, making the function reusable and easier to test. It also reduces the risk of unintended side effects caused by global state changes.

Overall, eliminating global dependencies leads to cleaner, more predictable, and maintainable code in real-world applications.

Task-02

Final Optimal Prompt:

the above given code refactor deeply nested if , elif, else logic
and maintain reliability , logical simplification, and maintainability

Code Screenshot :

```
14 def performance_label(score: int) -> str:
15     if score >= 90:
16         return "Excellent"
17     elif score >= 75:
18         return "Very Good"
19     elif score >= 60:
20         return "Good"
21     else:
22         return "Need Improvement"
23 score = 78
24 print(performance_label(score))
25
```

Output Screenshot:

```
● PS C:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant> c:; cd 'c:\U
xe' 'c:\Users\sakir\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\
yr 2sem\Ai-Assistant\lab13.py'
Very Good
○ PS C:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant>
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

This task highlights the importance of simplifying conditional logic to improve readability and maintainability. Deeply nested if-elif-else structures can make code harder to understand and debug. By refactoring the logic into a cleaner structure, such as using ordered conditions or mapping techniques, the code becomes more concise and easier to follow. This also improves scalability, as adding new conditions becomes simpler. Overall, reducing complexity in conditional statements enhances clarity, reduces errors, and results in more maintainable and professional code.

Task-03

Final Optimal Prompt:

Refactor the following Python code to remove repetition by using a reusable function and context managers (with open). Follow the DRY principle and improve code readability.

Code:

```
f = open("data1.txt")
print(f.read())
f.close()

f = open("data2.txt")
print(f.read())
f.close()
```

Code Screenshot :

```
40
41 def read_file(file_name):
42     with open(file_name, "r") as file:
43         print(file.read())
44
45 # Reuse function
46 read_file("data1.txt")
47 read_file("data2.txt")
```

Output Screenshot:

```
Hello Sabir,
Welcome to the Student Management System.
This file contains basic information.
Have a great day!
Course List:
1. Database Management
2. Operating Systems
3. Data Structures
4. Computer Networks

Keep learning and practicing!
PS C:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant>
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

This task demonstrates the importance of reducing code repetition by applying the DRY principle. The original code repeatedly opened, read, and closed files, which made it longer and harder to maintain. By introducing a reusable function and using context managers (with open), the code becomes cleaner and safer. The context manager automatically handles file closing, preventing resource leaks. This approach improves readability, reusability, and reliability, making the program more efficient and easier to scale when working with multiple files.

Task-04

Final Optimal Prompt:

Refactor the following Python code to improve search efficiency by using appropriate data structures like sets or dictionaries. Explain the improvement in time complexity.

Code:

```
users = ["admin", "guest", "editor", "viewer"]

name = input("Enter username: ")

found = False

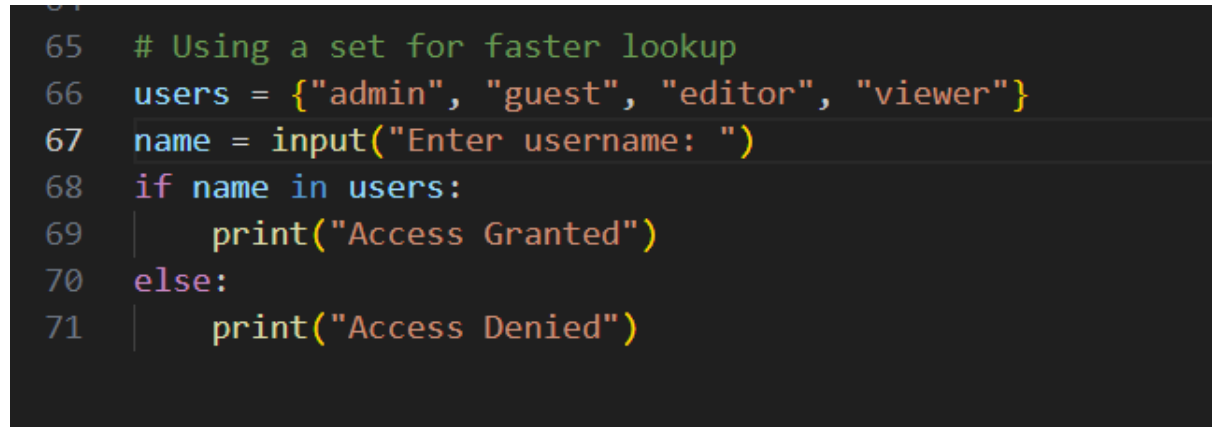
for u in users:

    if u == name:

        found = True

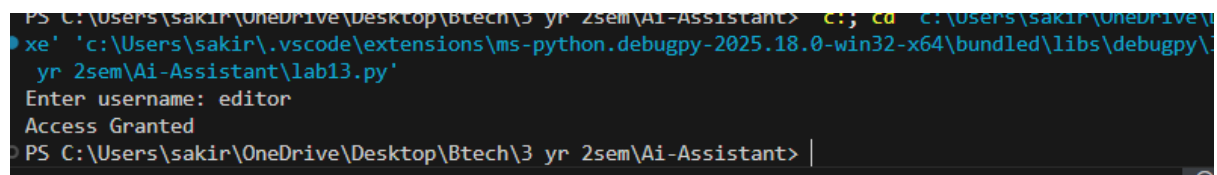
print("Access Granted" if found else "Access Denied")
```

Code Screenshot :

A screenshot of a code editor showing a Python script. The code defines a set of users and checks if an input username is in that set. The code is as follows:

```
65 # Using a set for faster lookup
66 users = {"admin", "guest", "editor", "viewer"}
67 name = input("Enter username: ")
68 if name in users:
69     print("Access Granted")
70 else:
71     print("Access Denied")
```

Output Screenshot:

A screenshot of a terminal window showing the execution of the Python script. The prompt is 'PS C:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant>'. The user enters 'editor' as the username, and the program outputs 'Access Granted'. The prompt is 'PS C:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant>'.

```
PS C:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant> cd C:\Users\sakir\OneDrive\
xe' 'c:\Users\sakir\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\
yr 2sem\Ai-Assistant\lab13.py'
Enter username: editor
Access Granted
PS C:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant> |
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

The search optimization task highlights the benefits of choosing appropriate data structures. The original code used a list and loop, resulting in linear time complexity $O(n)$. By replacing the list with a set, the lookup operation becomes significantly faster with average time complexity $O(1)$. This change improves performance, especially for large datasets. Additionally, the code becomes shorter and more readable by eliminating unnecessary loops. This task emphasizes how selecting the right data structure can greatly enhance efficiency and scalability in programs.

Tack-05

Final Optimal Prompt:

Refactor the following procedural Python code into an object-oriented design using a class. Apply encapsulation and create appropriate methods and attributes.

Code:

```
salary = 50000
tax = salary * 0.2
net = salary - tax
print(net)
```

Code Screenshot :

```

84
85 class SalaryCalculator:
86     def __init__(self, salary):
87         self._salary = salary          # encapsulated attribute
88         self._tax_rate = 0.2          # private tax rate
89     def calculate_tax(self):
90         return self._salary * self._tax_rate
91     def calculate_net_salary(self):
92         tax = self.calculate_tax()
93         return self._salary - tax
94     def display_net_salary(self):
95         net = self.calculate_net_salary()
96         print(net)
97 # Usage
98 obj = SalaryCalculator(50000)
99 obj.display_net_salary()

```

Output Screenshot:

```

PS C:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant> cd 'c:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant\lab13.py'
40000.0
PS C:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant>

```

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

This task illustrates how object-oriented programming improves code organization and reusability. The procedural code was simple but lacked structure and flexibility. By converting it into a class-based design, encapsulation is achieved, where data and related methods are grouped together. This allows better control over data and promotes modularity. Methods such as tax calculation and net salary computation make the logic reusable for different inputs. Overall, the OOP approach enhances maintainability, scalability, and readability, making the code more suitable for real-world applications.

Tack-06

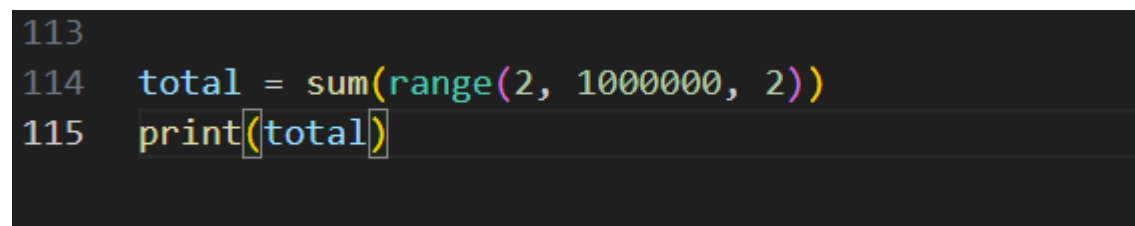
Final Optimal Prompt:

Optimize the following Python code to improve performance using better logic such as mathematical formulas or efficient Python constructs. Compare the performance improvement.

Code:

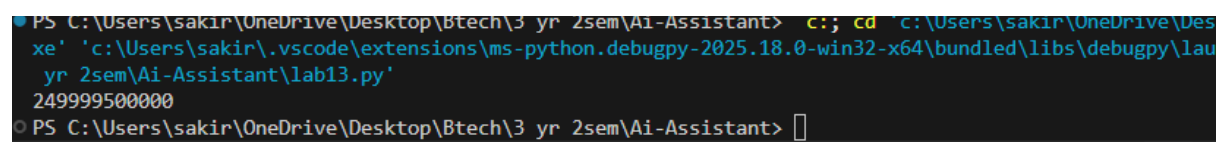
```
total = 0
for i in range(1, 1000000):
    if i % 2 == 0:
        total += i
print(total)
```

Code Screenshot :

A screenshot of a code editor showing three lines of Python code. Line 113 is empty. Line 114 contains 'total = sum(range(2, 1000000, 2))'. Line 115 contains 'print(total)'. The code is color-coded: 'total' is blue, 'sum' is green, 'range' is red, and 'print' is blue. The background is dark grey.

```
113
114 total = sum(range(2, 1000000, 2))
115 print(total)
```

Output Screenshot:

A screenshot of a terminal window showing the execution of a Python script. The prompt is 'PS C:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant>'. The command executed is 'c::; cd "C:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant\lab13.py"'. The output is '249999500000'. The prompt is 'PS C:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant>'.

```
PS C:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant> c::; cd "C:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant\lab13.py"
249999500000
PS C:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant>
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

The performance optimization task demonstrates how replacing iterative logic with mathematical formulas can significantly improve efficiency. The original code used a loop with condition checks, resulting in $O(n)$ time complexity. By using a direct formula to compute the sum of even numbers, the complexity is reduced to $O(1)$, making the execution almost instantaneous. This approach minimizes unnecessary computations and improves performance for large datasets. The task highlights the importance of understanding mathematical concepts to optimize algorithms effectively in programming.

Tack-07

Final Optimal Prompt:

Refactor the following Python code to remove side effects by avoiding modification of global variables. Use a functional approach where functions return values instead.

Code:

```
data = []  
  
def add_item(x):  
    data.append(x)  
  
add_item(10)  
add_item(20)  
  
print(data)
```

Code Screenshot :

```
130  
131 def add_item(data, x):  
132     return data + [x]  
133 data = []  
134  
135 data = add_item(data, 10)  
136 data = add_item(data, 20)  
137  
138 print(data)
```

Output Screenshot:

```
PS C:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant> cd 'c:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant'; & 'c:\p\xe' 'c:\Users\sakir\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '57224' '---' 'c:\Users\sakir\OneDrive\3 yr 2sem\Ai-Assistant\lab13.py'  
[10, 20]  
PS C:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant>
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

This task focuses on eliminating side effects by avoiding the modification of global variables. The original code directly changed a global list, making it harder to track changes and debug issues. By adopting a functional approach where functions return new values instead of modifying existing data, the code becomes more predictable and easier to test. This improves reliability and maintainability. Pure functions enhance code clarity and reduce unexpected behavior, making the program more robust and suitable for scalable and modular development.

Tack-08

Final Optimal Prompt:

Refactor the following Python code to improve readability and modularity by breaking complex validation into separate functions. Ensure the code is easy to test and understand.

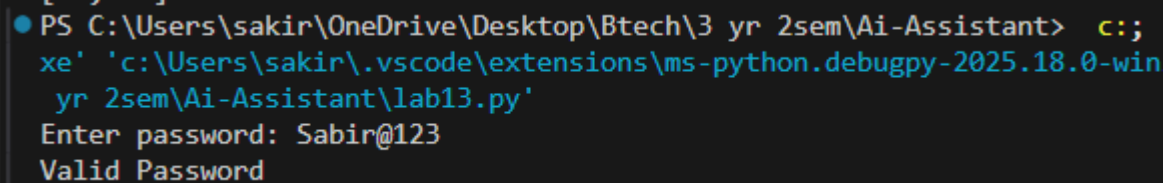
Code:

```
password = input("Enter password: ")
if len(password) >= 8:
    if any(c.isdigit() for c in password):
        if any(c.isupper() for c in password):
            print("Valid Password")
        else:
            print("Must contain uppercase")
    else:
        print("Must contain digit")
else:
    print("Password too short")
```

Code Screenshot :

```
157 def is_length_valid(password):
158     return len(password) >= 8
159 def has_digit(password):
160     return any(c.isdigit() for c in password)
161 def has_uppercase(password):
162     return any(c.isupper() for c in password)
163 def validate_password(password):
164     if not is_length_valid(password):
165         return "Password too short"
166     if not has_digit(password):
167         return "Must contain digit"
168     if not has_uppercase(password):
169         return "Must contain uppercase"
170     return "Valid Password"
171 # Main program
172 password = input("Enter password: ")
173 result = validate_password(password)
174 print(result)
```

Output Screenshot:



```
PS C:\Users\sakir\OneDrive\Desktop\Btech\3 yr 2sem\Ai-Assistant> c:;
xe' 'c:\Users\sakir\.vscode\extensions\ms-python.debugpy-2025.18.0-win
yr 2sem\Ai-Assistant\lab13.py'
Enter password: Sabir@123
Valid Password
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence) :

The input validation task emphasizes improving readability and modularity by breaking complex logic into smaller functions. The original code used deeply nested conditional statements, making it difficult to read and maintain. By separating validation checks into individual functions, the code becomes cleaner and easier to understand. Each function can be tested independently, improving reliability. This modular design enhances code reusability and simplifies debugging. Overall, the refactored approach promotes better coding practices and results in more maintainable and scalable programs.